

同濟大學

TONGJI UNIVERSITY

软件重构报告

课题名称	SmartSpar BikeHub 重构报告
副标题	智慧共享单车运营调度平台重构实践
小组名称	葫芦娃除BUG
日期	2026年6月15日

小组贡献说明

小组名称：葫芦娃除BUG

学号	姓名	团队定位	主要任务	贡献度
2352612	杜宇轩	组长	项目统筹、重构范围控制、运行验收协调、最终提交材料检查	20%
2352613	肖怡苒	组员	制定重构方案，梳理原项目问题、重构目标、测试计划和文档框架	20%
2351278	严梓杰	组员	后端重构：服务层抽取、统一响应、权限装饰器、路径配置和预测模块整理	20%
2351439	王梓彦	组员	前端重构：构建问题修复、运营中台化界面、设置页整合和地图体验优化	20%
2351583	马浩然	组员	测试与文档：MySQL 环境、pytest、前端构建验证、运行截图和测试报告整理	20%

目 录

1	重构总体思路	1
1.1	重构原因	1
1.2	重构目标与执行范围对照	1
1.3	重构计划	2
1.4	测试计划	2
2	高层重构：体系结构调整	4
2.1	后端分层调整	4
2.2	前端结构调整	5
2.3	运行架构	5
3	低层重构：代码味道、设计模式与 Bug 修复	7
3.1	统一响应与错误处理	7
3.2	角色权限装饰器	7
3.3	服务层抽取	8
3.4	路径配置集中化	8
3.5	预测模块降级启动	9
3.6	预测模型注册表	9
3.7	关键 Bug 修复	10
3.7.1	需求管理旧路由崩溃	10
3.7.2	Hook 规则修复	10
4	前端重构与界面修缮	11
4.1	构建基线修复	11
4.2	模板残留清理	11
4.3	运营中台化视觉	11
4.4	地图体验优化	14
5	测试报告	15

5.1 测试策略	15
5.2 测试环境	15
5.3 自动化测试.....	15
5.4 接口烟测	16
5.5 浏览器与预测验收	17
5.6 遗留风险与后续处理.....	17
6 源码、文档与贡献.....	18
6.1 重构后源码.....	18
6.2 文档清单	18
6.3 贡献说明	19
6.4 总结	19
附录：常用运行命令与验收清单.....	20

1 重构总体思路

1.1 重构原因

SmartSpar BikeHub 原项目已经覆盖共享单车调度平台的主要业务，但在工程质量和交付可复现性上存在以下问题：

- 本地启动依赖分散，前端、后端、MySQL、预测模块之间缺少统一说明。
- 后端部分路由函数过长，混合权限校验、参数解析、业务规则、数据库提交和响应拼装。
- 预测模型文件、上传目录和静态资源路径存在硬编码，Windows 本地环境下容易出错。
- 前端保留模板项目痕迹，登录、侧栏、仪表盘和表格页面风格不统一。
- TypeScript 构建、ESLint、pytest 和浏览器验收缺少稳定基线。

因此，本次重构的目标不是推倒重写，而是在保持现有 API 契约和业务功能的前提下，修复高影响问题，降低后续维护成本，并为最终提交准备完整材料。

1.2 重构目标与执行范围对照

为了避免重构范围失控，本次先将原重构方案中的目标拆分为可交付、可验证和可后续演进三类。可交付项优先处理会影响本地运行、构建、测试和演示验收的问题；可验证项要求有命令行输出、接口结果或页面截图支撑；范围较大的完整策略模式和值对象体系则记录为后续演进。表 1.1 汇总原计划关注点与本次执行结果。

表 1.1 原计划关注点与本次执行覆盖情况

关注点	原计划目标	本次执行证据	状态
重复代码	抽取权限、响应和错误处理公共逻辑	<code>response.py</code> 、 <code>decorators.py</code> 、统一错误装饰器	完成
过长方法	拆分胖路由，将业务规则移入服务层	<code>TaskService</code> 、 <code>DashboardService</code>	完成

关注点	原计划目标	本次执行证据	状态
路径硬编码	集中管理路径和环境差异	<code>Paths</code> <code>PredictionModelRegistry</code>	与 完成
霰弹式修改	新增模型时减少多处同步修改	轻量级预测模型注册表，训练策略 模式列为后续优化	部分完成
测试保护	用测试确认外部行为不变	<code>lint</code> 、 <code>build</code> 、 <code>pytest</code> 、API smoke、浏览器验收	完成
代码检查	记录缺陷、风险和验证结果	重构记录、测试报告、已知 warning 列表	完成
界面可用性	改善工作流和阅读体验	运营中台风格页面与截图证据	完成

1.3 重构计划

本次重构分为四个阶段：

（1）运行基线恢复：搭建本地 Node、Python、MySQL 环境，修复依赖安装、数据库初始化、测试账号、系统时间和预测数据问题。

（2）后端结构重构：抽取统一响应、角色权限装饰器、路径配置工具和服务层，减少路由层复杂度。

（3）前端体验重构：修复 TypeScript 构建，清理模板残留，调整界面为运营调度中台风格。

（4）测试与文档沉淀：执行 `lint`、`build`、`pytest`、浏览器验收和预测验收，整理运行手册、重构记录、测试报告和最终报告。

1.4 测试计划

测试采用命令行自动测试、接口烟测和浏览器验收结合的方式：

- 前端：执行 `npm run lint` 和 `npm run build`，并访问登录页、仪表盘、站点管理、调度管理、需求管理和设置页面。

- 后端：执行 `pytest`，覆盖 A* 路径规划、基础 API、数据库连接、用户搜索、聊天搜索、路由和集成测试。

- 集成：同时启动前后端，使用 `admin/admin123` 登录，验证核心业务页面。

- 预测：验证 DLinear、TiDE、TimesNet 标签可见，预测参数和 future 接口返回 200，页面正常渲染预测记录。

2 高层重构：体系结构调整

2.1 后端分层调整

原项目中，部分 Flask 路由函数承担了过多职责。以调度任务和仪表盘接口为例，路由函数中同时包含身份判断、角色判断、参数校验、业务计算、库存更新、数据库提交和响应格式处理。这种结构不利于测试和维护。

重构后，后端按如下职责划分：

- Route 层：负责 HTTP 请求入口、参数接收、权限入口和响应返回。
- Service 层：承载调度任务、仪表盘聚合等业务逻辑。
- Utils 层：提供统一响应、权限装饰器和错误处理。
- Config 层：集中管理模型 checkpoint、上传目录和静态资源路径。

重点新增模块包括：

- backend/app/utils/response.py
- backend/app/utils/decorators.py
- backend/app/config/paths.py
- backend/app/services/task_service.py
- backend/app/services/dashboard_service.py
- backend/app/services/prediction_registry.py

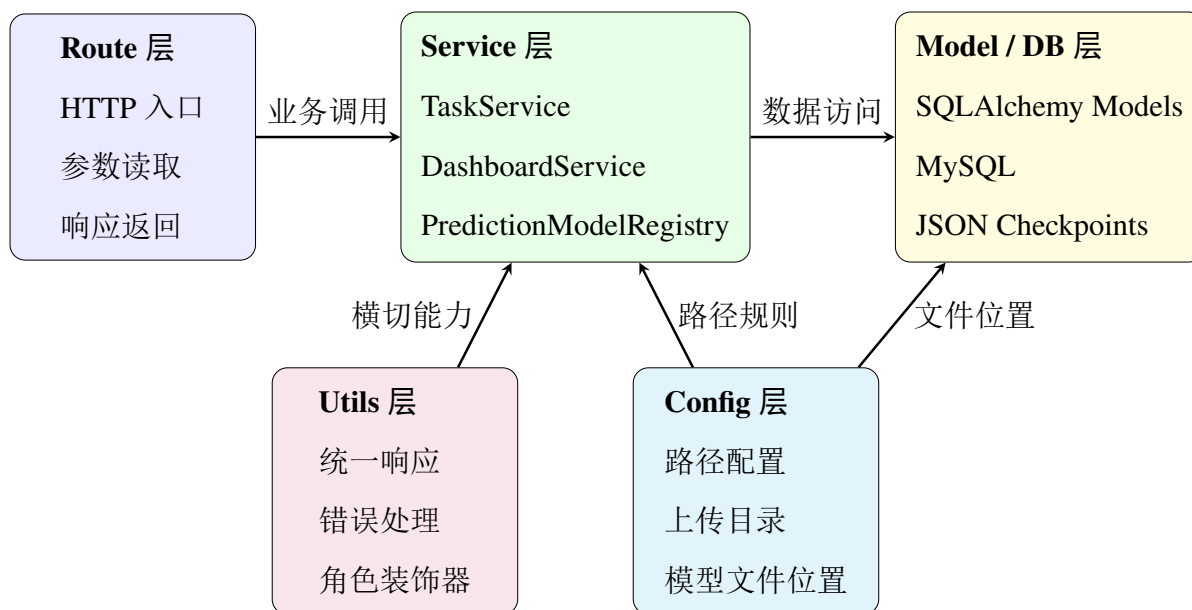


图 2.1 重构后的后端分层结构

2.2 前端结构调整

前端保留 React、TanStack Router、TanStack Query、TanStack Table 和 shadcn/ui 技术栈，但将模板后台改造为运营调度中台：

- 登录页和侧栏替换为 SmartSpar BikeHub 业务文案。
- 仪表盘突出任务数量、站点状态、预测需求和地图视图。
- 设置页面整合主题、侧栏、布局和阅读方向选项。
- 站点、任务、需求等表格页面统一搜索、筛选、按钮和表格密度。

2.3 运行架构

本地验证环境如下：

组件	地址或配置
前端 Vite 服务	http://127.0.0.1:5173/
后端 Flask 服务	http://127.0.0.1:5000/
MySQL 本地实例	127.0.0.1:3307
开发数据库	bikehub_test_dev

测试数据库

bikehub_test_test

本地运行手册见 docs/LOCAL_RUNBOOK.md。

3 低层重构：代码味道、设计模式与 Bug 修复

3.1 统一响应与错误处理

重构前，不同接口自行拼装 JSON 响应，错误字段不统一，前端只能显示泛化错误信息。重构后新增统一响应工具：

- success_response
- error_response
- 分页响应工具
- 通用路由错误处理装饰器

该改动减少了重复代码，使前端可以稳定读取后端返回的 error 或 message 字段。

```

1  def success_response(data=None, message="success", status_code
    =200, **extra):
2      payload = {"success": True, "message": message}
3      if data is not None:
4          payload["data"] = data
5          payload.update(extra)
6      return jsonify(payload), status_code
7
8  def error_response(message, status_code=400, error_code=None, **
    extra):
9      payload = {"success": False, "error": message}
10     if error_code:
11         payload["error_code"] = error_code
12     payload.update(extra)
13     return jsonify(payload), status_code

```

代码 3.1 统一响应工具片段

3.2 角色权限装饰器

原项目中，管理员、调度员、运维人员的角色判断散落在多个路由中。重构后新增角色装饰器：

- require_roles

- `require_admin`
- `require_dispatcher_or_admin`

通过装饰器将权限逻辑集中维护，避免不同接口权限判断不一致。

```
1 @users_bp.route("/users", methods=["GET"])
2 @jwt_required()
3 @require_admin
4 @handle_route_errors
5 def get_users():
6     users = User.query.all()
7     return success_response([user.to_dict() for user in users])
```

代码 3.2 角色权限装饰器使用方式

3.3 服务层抽取

`dispatch_tasks.py` 原本存在典型胖路由问题，任务创建、更新、删除、派发和库存历史维护都放在路由函数中。重构后新增 `TaskService`，将调度任务业务规则从路由层移出。

类似地，仪表盘聚合逻辑迁移到 `DashboardService`，负责聚合站点、预测、库存和任务状态数据。这样 `Route` 层只负责请求响应，业务规则更容易测试和复用。

3.4 路径配置集中化

预测文件路径、上传路径和静态资源路径原来存在硬编码。重构后新增 `backend/app/config/paths.py`，统一管理：

- 时间序列 `checkpoint` 路径。
- 上传目录。
- 静态资源目录。

该改动减少了相对路径错误，也方便在 Windows 本地和最终提交环境之间迁移。

3.5 预测模块降级启动

原后台 scheduler 在缺少 torch 时可能导致 Flask 无法启动。重构后改为可选依赖降级：

- Web 服务可以在未安装深度学习栈时先启动。
- 缺少训练依赖时跳过预测训练任务，并记录清晰日志。
- 已生成的 future JSON 仍可被预测接口读取并展示。

3.6 预测模型注册表

原重构方案指出，新增预测模型时容易同时修改路由、调度器、训练逻辑和前端组件，属于典型的霰弹式修改。本次重构在保持接口兼容的前提下补充了轻量级模型注册表 PredictionModelRegistry：

- 集中维护 DLinear、TiDE、TimesNet 等模型的名称和文件位置。
- 预测参数和 future 数据接口不再自行拼接模型文件路径，而是通过注册表获取模型规格。
- 调度器默认运行的模型列表由注册表提供，后续新增模型时首先修改注册表。
- 新增 /api/predictions/models 接口，便于前端或验收脚本读取当前支持的模型列表。

由于项目周期有限，本次没有完整重写深度学习训练代码为策略模式或工厂模式；但注册表已经把最容易散落的模型清单和路径规则收拢，降低了后续继续演进为完整策略模式的成本。

```
1 @dataclass(frozen=True)
2 class PredictionModelSpec:
3     name: str
4     family: str = "time_series"
5
6 class PredictionModelRegistry:
```

```

7     def __init__(self, models=None):
8         self._models = {model.name: model for model in models or
                           DEFAULT_MODELS}
9
10    def names(self):
11        return list(self._models.keys())
12
13    def require(self, name):
14        if name not in self._models:
15            raise ValueError(f"Unsupported prediction model: {name
                               }")
16        return self._models[name]

```

代码 3.3 预测模型注册表片段

3.7 关键 Bug 修复

3.7.1 需求管理旧路由崩溃

修复前，访问 `/demand_management` 会触发 TanStack Router 的 active match 错误，页面进入错误边界。

原因是项目中同时存在下划线和短横线两套路由，组件使用的是短横线路由 API。修复后，旧路径作为兼容入口跳转到 `/demand-management`。

3.7.2 Hook 规则修复

修复内容包括：

- 将 `useConfirm` 调用限制在 React 组件顶层。
- WebSocket 重连逻辑使用 `ref` 保存连接函数，避免闭包和 hook 规则问题。
- 输入状态防抖改为 `useRef + useCallback`。
- 地图初始化函数改为 `useCallback` 并在 `effect` 前定义。

修复后，`npm run lint` 不再存在阻塞 error。

4 前端重构与界面修缮

4.1 构建基线修复

前端首先恢复 TypeScript 构建基线，主要处理：

- TanStack Table ColumnMeta.title 类型补充。
- 表单 resolver 类型兼容。
- 数字和字符串筛选值统一。
- 未使用模板代码对构建的阻塞。

最终 `npm run build` 已通过。

4.2 模板残留清理

原项目保留了模板后台中的默认用户、示例团队和英文文案。重构后：

- 侧栏文案改为 BikeHub 业务模块。
- 登录页改为中文业务文案。
- 默认用户和团队信息替换为运营调度平台信息。
- 主题设置从浮动抽屉融入正式设置页面。

4.3 运营中台化视觉

前端视觉调整遵循“运营中台”原则：克制背景、清晰侧栏、紧凑头部、统一表格和卡片间距。重点优化页面包括：

- 仪表盘：增加任务统计、站点排行、预测需求和地图视图。
- 站点管理：统一搜索、筛选、列视图和表格布局。
- 调度管理：展示任务状态、优先级和操作入口。

- 需求管理：提供实时需求和预测需求两个视图。
- 设置页面：集中管理个人资料、主题、侧栏、布局和阅读方向。



图 4.1 运营总览页面：任务统计、站点排行、系统时间和地图降级视图



图 4.2 站点管理页面：统一搜索、导入、创建、列视图和表格布局



图 4.3 需求预测页面：DLinear 模型 future 数据正常展示



图 4.4 设置页面：主题、侧栏、页面布局和阅读方向已融入正式设置

4.4 地图体验优化

仪表盘地图使用高德地图。考虑到本地网络、Key 或安全码可能不可用，重构中增加了 fallback:

- 高德底图可用时展示真实地图。
- 高德底图不可用时切换为本地站点分布图。
- 地图容器采用稳定高度和边框，避免空白区域影响页面观感。

5 测试报告

5.1 测试策略

测试目标是证明重构没有破坏核心外部行为，并且本地提交版本具备可复现的运行、构建和验收流程。本次测试分为三层：

- 工程基线测试：确认前端 lint、生产构建和后端 pytest 可以稳定执行。
- 接口烟测：确认登录、系统时间、预测模型列表、预测参数和 future 数据接口可用。
- 浏览器验收：确认管理员登录后可以访问仪表盘、站点管理、调度管理、需求管理和设置页面。

原始命令输出保存在 docs/refactoring_report_tex/evidence/，截图证据保存在 docs/refactoring_report_tex/figures/evidence/。

5.2 测试环境

项目	配置
前端地址	http://127.0.0.1:5173/
后端地址	http://127.0.0.1:5000/
MySQL	127.0.0.1:3307
管理员账号	admin/admin123
调度员账号	dispatcher/dispatcher123
运维账号	operator/operator123

5.3 自动化测试

测试项	命令	结果
前端 Lint	npm run lint	通过, 0 errors, 保留 warnings
前端构建	npm run build	通过, 存在大 chunk 警告
后端测试	python -m pytest	20 passed, 4 skipped

后端新增的重构工具测试覆盖统一响应、错误装饰器、角色装饰器、路径配置和预测模型注册表。前端 lint 当前已经没有阻塞 error，剩余 warning 主要来自历史模板页面、any 类型和部分 hook 依赖提示，记录为后续低风险清理项。

5.4 接口烟测

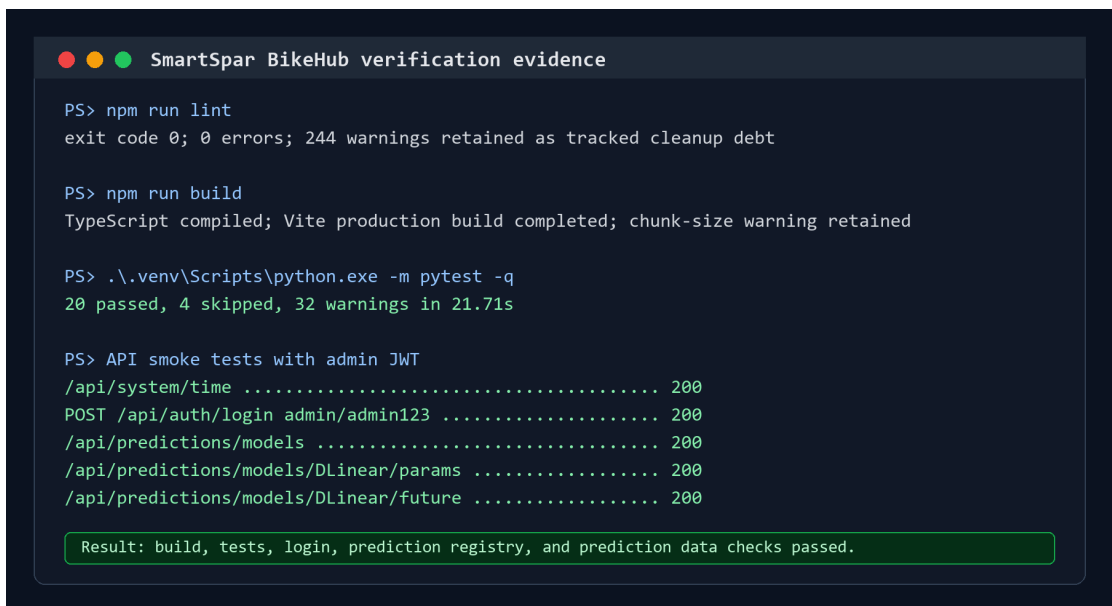
系统时间接口：

GET `http://127.0.0.1:5000/api/system/time`

返回示例：

```
{
  "current_time": "2026-06-15T14:50:11",
  "mode": "real",
  "timezone": "Asia/Shanghai"
}
```

登录接口使用 `admin/admin123` 可以返回 access token，前端可进入受保护页面。



```
PS> npm run lint
exit code 0; 0 errors; 244 warnings retained as tracked cleanup debt

PS> npm run build
TypeScript compiled; Vite production build completed; chunk-size warning retained

PS> .\.venv\Scripts\python.exe -m pytest -q
20 passed, 4 skipped, 32 warnings in 21.71s

PS> API smoke tests with admin JWT
/api/system/time ..... 200
POST /api/auth/login admin/admin123 ..... 200
/api/predictions/models ..... 200
/api/predictions/models/DLinear/params ..... 200
/api/predictions/models/DLinear/future ..... 200

Result: build, tests, login, prediction registry, and prediction data checks passed.
```

图 5.1 命令行验证结果摘要

预测相关接口烟测结果如下：

接口	状态	验证目的
/api/predictions/models	200	模型注册表可读
/api/predictions/models/DLinear/params	200	参数文件可读
/api/predictions/models/DLinear/future	200	future 分页数据可读

5.5 浏览器与预测验收

验收项	验收结果	状态
登录页	管理员账号 <code>admin/admin123</code> 可登录，登录后进入受保护页面	通过
仪表盘	展示运营总览、任务统计、系统时间、地图区域和站点排行	通过
站点管理	展示演示站点，搜索、列视图和表格布局可用	通过
调度管理	展示演示调度任务，状态和操作入口可见	通过
需求管理	实时需求记录可见，旧路径 <code>/demand_management</code> 自动跳转	通过
设置页面	外观、侧栏、布局和阅读方向配置已融入正式设置页	通过
预测功能	<code>DLinear</code> 、 <code>TiDE</code> 、 <code>TimesNet</code> 标签可见， <code>DLinear future</code> 数据可渲染	通过

5.6 遗留风险与后续处理

风险项	后续处理建议	影响
前端 lint 仍有 warning	按模块清理历史模板残留、 <code>any</code> 类型、未使用 <code>import</code> 、 <code>console</code> 和 <code>hook</code> 依赖提示	低
Vite 提示部分 chunk 大于 500 KB	通过路由懒加载和 <code>manual chunks</code> 优化首屏资源	低
SQLAlchemy relationship overlap warning	通过 <code>back_populates</code> 、 <code>viewonly</code> 或 <code>overlaps</code> 修正模型关系	中
重新训练预测模型需要 torch 环境	将训练环境与演示运行环境分离，提交说明中保留可选安装步骤	中

6 源码、文档与贡献

6.1 重构后源码

重构后源码仓库链接:

https://github.com/dyx131313/SmartSpar_BikeHub

主要目录:

- 后端入口: `backend/run.py`
- 后端应用: `backend/app`
- 后端服务层: `backend/app/services`
- 后端工具: `backend/app/utils`
- 前端入口: `frontend/bikehub/src`
- 前端页面: `frontend/bikehub/src/features`

6.2 文档清单

本次交付文档包括:

- `docs/LOCAL_RUNBOOK.md`: 本地运行手册。
- `docs/REFACTORING_RECORD.md`: 重构过程记录。
- `docs/TEST_REPORT.md`: 测试报告。
- `docs/REFACTORING_REPORT_DRAFT.md`: Markdown 版报告草稿。
- `docs/refactoring_report_tex/`: 基于同济 TeX 模板的正式报告工程。
- `docs/refactoring_report_tex/figures/evidence/`: 页面截图、架构图、覆盖矩阵和命令行验证图。
- `docs/refactoring_report_tex/evidence/`: lint、build、pytest、接口烟测的原始输出。

6.3 贡献说明

小组名称为“葫芦娃除 BUG”。成员分工和贡献度已单独整理在封面后的“小组贡献说明”页，作为报告第二页展示。

6.4 总结

本次重构将 SmartSpar BikeHub 从一个可以演示但工程基线不稳定的项目，整理为可以本地启动、可以构建、可以测试、可以说明的提交版本。后端通过服务层、权限装饰器、统一响应和路径配置降低了维护复杂度；前端通过构建修复、运营中台化设计和关键路由修复提升了可用性；测试和文档则保证重构结果可复现、可验收。

附录：常用运行命令与验收清单

常用运行命令

```
cd frontend\bikehub

npm ci

npm run dev -- --host 127.0.0.1 --port 5173

npm run lint

npm run build

cd backend

.\.venv\Scripts\python.exe -m flask --app run.py run `
    --host 127.0.0.1 --port 5000 --debug

.\.venv\Scripts\python.exe -m pytest
```

本地账号与接口

项目	内容
前端地址	http://127.0.0.1:5173/
后端地址	http://127.0.0.1:5000/
管理员账号	admin/admin123
调度员账号	dispatcher/dispatcher123
运维账号	operator/operator123
系统时间接口	/api/system/time
预测模型接口	/api/predictions/models
预测数据接口	/api/predictions/models/DLinear/future

提交材料检查清单

材料	位置或说明	状态
重构后源码	项目根目录，包含前端、后端和测试代码	已准备
运行手册	docs/LOCAL_RUNBOOK.md	已准备
重构记录	docs/REFACTORING_RECORD.md	已准备
测试报告	docs/TEST_REPORT.md 与本文第 5 章	已准备

正式报告	docs/refactoring_report_tex/main.pdf	已准备
命令行证据	docs/refactoring_report_tex/evidence/	已准备
页面截图	figures/evidence/, 包含登录、仪表盘、站点、需求 and 设置页截图	已准备
源码仓库链接	https://github.com/dyx131313/SmartSpar_BikeHub	已准备

已知限制

- 高德地图依赖网络、Key 和安全码，本地不可用时会自动切换为站点分布图。
- 前端 lint 当前为 0 errors，但仍保留历史 warning，后续可按模块逐步清理。
- 重新训练深度学习预测模型需要单独准备 torch 相关环境；本次验收重点为已生成预测数据的读取和展示。